Apartment 50: Paper 007

AuthorityEffect: None. This is fictional examination staging, not a historical quotation or a source of mathematical authority.

Landau points at the declared carrier. Woland asks which claim its checker can establish. Behemoth asks for the smallest valid near miss. Their task in this companion is **prediction, execution, and learning evidence**.

Begin with the study guide, then attempt the 24-problem quiz before opening the separate instructor key. Record each first prediction before viewing reference outputs. A lower-case wire token names a role only after its interpretation is declared. Arrows sketch graph edges; they do not automatically attach higher-dimensional cells. A receipt records its declared check and inputs. It cannot establish learner transfer without learner observations.

The examiner's standard is a concrete calculation, an explicit counterexample, or a source-bound missing obligation. The manuscript source reading governs the answer.

Paper 007: Prediction, execution, and learning evidence

Study guide: Paper 007

AuthorityEffect: None. This guide prepares the quiz; the instructor key is a separate file. The preface uses fictional examiners. The mathematics and evidence come from the paper, the local notebook, and explicitly declared examples.

You need integer arithmetic, fractions, and a way to read a directed edge. Study the complete calculation, cover a step and reconstruct it, then predict a changed instance before running code. Keep the first response and any later correction. The eight levels below describe an instructional sequence, not an empirically calibrated theory of learning.

Levels 0–1: One carrier and a complete calculation

A graph carrier declares vertices and edges. This triangle has ordered edges ab, bc, ca, rational coefficients, and no filled face. The arrow from s to t means that the difference is $f_t - f_s$. A **claim table** h assigns one rational number to each directed edge; a **potential** f assigns one to each vertex. The residual is $r = h - df$.

Edge	Claim h	Source height	Target height	Difference df	Residual r
ab	$1/3$	0	$1/3$	$1/3$	0
bc	$1/3$	$1/3$	$2/3$	$1/3$	0
ca	$1/3$	$2/3$	0	$-2/3$	1
Signed sum	1	—	—	0	1

The difference moves the discrepancy onto one edge. It cannot make all three residual coordinates zero because every potential difference sums to zero around this cycle, while the claims sum to one. Here “sum” means the signed sum in

the declared order. It is not a norm, a weighted optimization result, or a native solver certificate.

Use `Fraction(1, 3)` or `Fraction("1/3")` to specify 1/3 directly. In `Fraction(1/3)`, Python first performs floating-point division. The resulting fraction exactly represents that float, which can differ from 1/3. Exact arithmetic does not repair an input that already represents a different number. The worksheet below does not include a general input validator.

Try before checking: add the same rational k to every height. On every edge,

$$(f_t + k) - (f_s + k) = f_t - f_s.$$

Therefore the differences and residual stay fixed. Negating the claims while keeping the potential gives $-h - df$, not $-(h - df)$. Write out both expressions to locate the sign difference.

Level 2: A proof, then a changed instance

For any directed closed walk $v_0, \ldots, v_m = v_0$,

$$\sum_{i=0}^{m-1} (df)_{v_i v_{i+1}} = \sum_{i=0}^{m-1} (f_{v_{i+1}} - f_{v_i}) = 0.$$

Each height occurrence cancels with the adjacent occurrence, including at the closing edge. Repeated vertices do not invalidate this telescoping calculation. It follows algebraically that $C(h - df) = C(h)$ on the walk. This is a general identity under the stated definitions; checking a few potentials is a finite illustration of it.

For this particular triangle, a zero cycle sum also lets us construct a potential: set $f_a = 0$, $f_b = h_{ab}$, $f_c = h_{ab} + h_{bc}$. The closing difference is then $-h_{ab} - h_{bc}$, equal to h_{ca} exactly when the sum is zero. Do not silently extend this single-cycle criterion to arbitrary graphs without checking every relevant cycle.

A path has no closing constraint. With path claims $(1, 2)$, heights $(0, 1, 3)$ realize both edges. Closing it with claim 1 creates residual $(0, 0, 4)$. Changing the closing claim to -3 changes the claim table and makes it exact. Attaching a filled cell changes the carrier and adds further semantics; the drawing alone performs neither operation.

Completion practice: cover the closing claim, derive its value, and label explicitly whether you changed a claim, a potential, or the carrier.

Level 3: Make the checker discriminate

A useful control is a plausible implementation fault with a known consequence. Reversing the difference gives $\widetilde{df} = (-1/3, -1/3, 2/3)$. Its signed sum is still zero and the wrong residual $(2/3, 2/3, -1/3)$ still sums to one. A cycle-sum

checker alone cannot discriminate the sign convention. The independently worked coordinate table can.

Different controls check different claims. A parser needs an invalid-syntax example. A digest check needs changed content bound to an unchanged digest. A finite LP certificate needs computed objectives **and** primal/dual feasibility. For minimize nonnegative x with $x \geq 1$, and dual maximize y with $0 \leq y \leq 1$, the pair $x = y = 0$ has equal objectives but an infeasible primal. The pair $x = y = 1$ satisfies both feasibility and equality.

The legacy status-grading example scans for the substring `assert`. A comment or string containing that word can enter its denominator. It reads supplied statuses and does not execute code. Even a more careful assertion count would need a declared scoring interpretation; assertions are not participant answers.

Practice: say which property your control can falsify, why it should fail, and which faults it could still miss.

Level 4: Keep source identity and execution separate

Evidence	Supports	Leaves open
Stored outputs and execution counts	The deposited JSON contains those fields	Whether a cell executed now, or a person predicted it
Fresh isolated notebook replay	These cells execute in the declared process/backend; matching outputs reproduce this reference run	General theorem, learner mastery, full repository integration
Hash and file-census verification	Listed bytes and archive contents agree	Mathematical truth, pedagogical quality
Historical source-bound receipt	The recorded outcome for its stated inputs and environment	Outcome after source or API changes
Source inspection	What the inspected version declares or implements	Fresh execution and uninspected dependencies

From the package root, install the declared development dependencies, then replay:

```
python -m pip install -e '.[dev]'
python tools/replay_companion_notebook.py
python -m pytest code/tests -q -ra
```

The default replay executes ordinary Python cells in a fresh CPython process and compares sequential cell counters and normalized outputs without writing executed results into the deposited notebook. The counters are not Jupyter kernel history. An optional `--backend jupyter --kernel NAME` replay requires a registered kernel. That backend could not run on the review host because sockets were unavailable; CPython success does not establish Jupyter rendering or IPython behavior. It verifies that the file did not change during replay. The deposit's stored outputs may be updated intentionally when authoring a new candidate, followed by a new receipt and manifest.

The source inventory pins inspected sources and the 20 route notebooks at audit commit `52cb5457f6c485ba548cfd292f6012a941ed22cf`. If a cited file moves, obtain the original commit's bytes and check identity, or record that they are unavailable. A different current implementation needs its own labeled run. A same-name file is not a historical substitute.

The saved 20/20 route receipt has old paths; a route notebook also uses a retired norm API. The local fraction lesson avoids that interface. Its success does not refresh the external route.

Level 5: Read every formal hypothesis

The list bound concerns an integer list and a natural number n:

$$\operatorname{mass}(h) = \sum_{x \in h} |x|, \quad |h| \le n, \quad |x| \le n - 1 \quad \Longrightarrow \quad \operatorname{mass}(h) \le S(n).$$

Here $S(n) = 0$ for $n < 2$, otherwise $n(n-1)$; subtraction on natural numbers truncates at zero. At $n = 3$, the maximum follows by counting at most three entries of magnitude at most two. A magnitude cap without a length bound cannot supply this result. These are declared finite mathematical conditions, not assumptions validated for human minds.

The unary nesting lemma preserves supplied symbol/operator declaration predicates when the nested operator is allowed and the starting term satisfies the predicates. It permits arbitrary finite nesting depth; it does not impose a maximum depth or a search-time bound.

The named report has 21 selected declarations. The full package-local report has 64, including generated/private declarations, and excludes imported library declarations. Distinguish permitted axioms from observed dependencies. Reading a saved report is not rerunning Lean.

Practice: for each theorem state the carrier, assumptions, conclusion, and a tempting interpretation it does not support.

Level 6: Record what a learner did

A code result and a learner result have different units of observation. A learner assessment needs the person's first response to a task under a stated exposure

condition and an explicit scoring rule. A proposed minimal record is:

Field	Purpose
Participant pseudonym, task/version, source hashes	Identify the learner observation and task bytes
Baseline task response and assigned condition	Record prerequisites and comparison
First prediction, explanation, submission/exposure order	Distinguish prediction from answer reproduction
Feedback shown and corrected response	Retain learning from feedback without replacing the first response
Novel transfer task, response, independent score	Assess a prespecified skill on an unfamiliar instance
Delayed response or explicit missing value/reason	Preserve the denominator and missingness

For a proposed study, define the compared lesson formats, assignment procedure, assessment tasks, scoring, exclusions, timing, missing-response handling, and analysis before inspecting results. Use tasks whose answers are not supplied by the lesson, and retain the number assigned and assessed in each condition. A corrected response after seeing the key is not a first prediction. A missing response is not automatically a zero or a pass; use the prespecified analysis and disclose missingness.

These requirements outline a future investigation. This package has no participant corpus, no measured attention half-life, and no estimate of transfer effect.

Level 7: Write the strongest supported sentence

Build a lesson with a declared example, visible derivation, first prediction, changed-input probe, and boundary. State what was actually checked: "The declared rational triangle reproduces the displayed residual and rejects the deliberate wrong-sign implementation by its coordinates" is assessable. "All students transfer cycle reasoning" needs human observations and a study design beyond the cells.

For publication, distinguish standalone tests, isolated source-module checks, a full repository run, formal-kernel replay, and historical records. A local release archive is a publication candidate; a verified archive and a proposed grading rubric do not create a DOI or certify a learner.

Exact worksheet

Save the following as a scratch Python file, or run it in a fresh cell. Predict the rows on paper first. It imports only the standard library and checks arithmetic

and a deliberate mutant. It contains reference answers; running it is not an assessment of the reader.

```
from fractions import Fraction as Q

edges = (("a", "b"), ("b", "c"), ("c", "a"))
h = (Q(1, 3), Q(1, 3), Q(1, 3))
f = {"a": Q(0), "b": Q(1, 3), "c": Q(2, 3)}
df = tuple(f[t] - f[s] for s, t in edges)
r = tuple(claim - delta for claim, delta in zip(h, df))
assert df == (Q(1, 3), Q(1, 3), Q(-2, 3))
assert r == (Q(0), Q(0), Q(1))
assert sum(df, Q(0)) == 0 and sum(r, Q(0)) == 1

shifted = {v: height + Q(7, 5) for v, height in f.items()}
assert tuple(shifted[t] - shifted[s] for s, t in edges) == df
negated = tuple(-claim - delta for claim, delta in zip(h, df))
assert negated == (Q(-2, 3), Q(-2, 3), Q(1, 3))

wrong_df = tuple(f[s] - f[t] for s, t in edges)
wrong_r = tuple(claim - delta for claim, delta in zip(h, wrong_df))
assert sum(wrong_df, Q(0)) == 0 and sum(wrong_r, Q(0)) == 1
assert wrong_df != df
assert wrong_r == (Q(2, 3), Q(2, 3), Q(-1, 3))

completed = (h[0], h[1], -h[0] - h[1])
assert completed == df
assert Q(1, 3) == Q("1/3") and Q(1 / 3) != Q(1, 3)
assert sum(abs(x) for x in (2, -2, 1)) == 5 <= 3 * (3 - 1)
print("triangle, changed-input probes, and wrong-sign control pass")
```

After feedback, create a different cycle and give its orientation, claims, potential, predicted residual, and evidence sentence. Revisit levels 0–2 for arithmetic/scope errors, levels 3–5 for checker/source/formal errors, and levels 6–7 for assessment/publication errors. Retain the first and revised attempts. Read the instructor key only after completing the corresponding quiz.

Paper 007: Prediction, execution, and learning evidence

Instructor key: Paper 007

AuthorityEffect: None. This proposed rubric accompanies the quiz and study guide. It is not a validated certification standard. Give one point for each numbered scoring item below: four per problem, 96 total. Accept equivalent correct constructions and clearly declared conventions. Record missing or unsupported steps without treating a completed correction as the first response.

Level 0

1. (1) Vertices a, b, c and ordered edges ab, bc, ca. (2) Coefficients are rational; claims and heights have separate roles. (3) $df_{st} = f_t - f_s$. (4) No filled face follows from the drawing; adding an edge or face changes the carrier.

2. (1) `Fraction(1,3)` specifies 1/3. (2) `Fraction("1/3")` specifies the same rational. (3) `Fraction(1/3)` first converts the result of floating-point division. (4) It represents that float exactly, not automatically the intended rational 1/3; use an integer numerator/denominator or exact string for this example.

3. (1) Declare the carrier and show a worked calculation. (2) Ask for a prediction with a reason before output exposure. (3) Change one input and compare with an independently derived reference. (4) Retain the first response and state the evidence boundary; stored reference answers do not record learner predictions.

Level 1

4. (1) $df = (1/3, 1/3, -2/3)$. (2) $r = (0, 0, 1)$. (3) Sums are $1, 0, 1$, respectively. (4) The original claims cannot all equal potential differences on the cycle; the nonzero sum is the obstruction under this convention.

5. (1) Shifted heights $7/5, 26/15, 31/15$. (2) Differences unchanged. (3) Residual unchanged. (4) The same shift is subtracted at each edge's endpoints: $k - k = 0$.

6. (1) Residual $(-2/3, -2/3, 1/3)$. (2) Sum -1. (3) It differs from $-r = (0, 0, -1)$. (4) Claims changed sign while the potential did not; generally $-h - df \neq -h + df$.

Level 2

7. (1) Expand the endpoint-difference sum. (2) Cancel successive height occurrences and use $v_m = v_0$. (3) Subtract zero to obtain $C(h - df) = C(h)$ under the stated rational definitions. (4) A finite list of tested potentials illustrates, rather than proves, the general identity.

8. (1) Third claim $-2/3$. (2) Completed claims sum to zero. (3) $f = (0, 1/3, 2/3)$ realizes them, so the completed residual is zero. (4) Choosing/replacing a claim changes h, whereas a potential adjustment on the original h preserves its nonzero sum.

9. (1) The path has zero residual with $f = (0, 1, 3)$. (2) Adding closing claim 1 gives difference -3 and residual $(0, 0, 4)$, sum 4. (3) Closing claim -3 makes the table exact. (4) A face is a separate carrier declaration and cannot silently alter the reports or the potential-difference identity.

Level 3

10. (1) $\widetilde{df} = (-1/3, -1/3, 2/3)$. (2) Wrong residual $(2/3, 2/3, -1/3)$. (3) Both stated sum checks pass. (4) Compare predicted differences with $(1/3, 1/3, -2/3)$, or compare the residual with $(0, 0, 1)$; this detects the specific sign fault.

11. (1) Computed objectives for $x = y = 0$ are both zero. (2) $x = 0$ violates $x \geq 1$. (3) Check primal $x \geq 1$, dual $0 \leq y \leq 1$, and objective values from the supplied witnesses before using equality. (4) $x = y = 1$ is feasible on both sides and has equal objective 1; this declared LP has optimum 1.

12. (1) All three sources contain the substring `assert`. (2) Two supplied statuses are `"ok"`, so the reported grade is 2/3. (3) Comments and strings are counted although neither is an assertion. (4) The function trusts supplied fields, executes no code, and observes no learner response. Either of these last two distinctions must be explained, not merely named.

Level 4

13. (1) Reading JSON does not execute the notebook. (2) Execute all ordinary Python cells in order in a fresh CPython process, or start an explicitly selected Jupyter kernel; identify which backend actually ran. (3) Compare code-cell counts and normalized output data with the deposited reference without overwriting it; retain commands, versions, hashes, and failures; CPython counters do not demonstrate Jupyter kernel history. (4) A match does not establish learner mastery, a full route pass, or a theorem beyond the stated computation.

14. (1) Obtain commit A's source bytes and verify their identity. (2) A current-B replay may be useful only as a separately labeled check of B. (3) Never silently substitute B as evidence for A. (4) If A is unavailable, record revision/path, expected identity, attempted command and unavailable dependency, with historical replay unavailable/not run; do not call it a successful historical replay.

15. (1) The receipt supports its recorded dated run and exact scope, subject to its provenance. (2) Reconcile current route/manifest/source paths. (3) Replace retired APIs with maintained operations and generate a new source-bound route receipt. (4) The fraction-only notebook exercises a different standalone path and cannot clear the external route blockers.

Level 5

16. (1) Mass $2 + 2 + 1 = 5$, ceiling 6, with both hypotheses satisfied. (2) $(2, -2, 2)$ has mass 6 and saturates the bound. (3) At $n = 0$, length at most zero forces the empty list and mass zero. (4) At $n = 1$, the entry cap is zero, so every permitted entry is zero and the mass is zero.

17. (1) For example (7) has length one but violates the cap and has mass 7. (2) $(2, 2, 2, 2)$ respects the cap but violates length, with mass 8. (3) The implication does not apply when a hypothesis is false. (4) No measured mapping from human cognition to these integer-list conditions is supplied.

18. (1) Twenty-one counts named selected declarations; 64 counts all declarations defined in the three package-local modules, including generated/private names. (2) Imported Lean library declarations remain outside that census, and whitelist membership is not usage; recorded rows list no `Classical.choice` dependency. (3) Saved output does not establish a fresh kernel replay on this host; bind a successful run to the actual formal bytes. (4) Allowed unary nesting preserves supplied declaration predicates at any finite depth and supplies no maximum depth or student search-time bound.

Level 6

19. (1) $92/100 \geq 9/10$. (2) $88/100 \geq 4/5$ and $85/100 \geq 4/5$. (3) Values and thresholds are supplied fixtures; no operational definitions, participants, task responses, or sampling are supplied. (4) The test verifies comparison logic, not a measured learner pass rate or validation of these thresholds.

20. (1) Obtain a predefined baseline measure of relevant skill. (2) Assign comparable conditions, preferably using a stated randomized procedure, while controlling task/feedback exposure. (3) Independently score prespecified novel and delayed tasks using retained responses and denominators. (4) Disclose missingness and exclusions; seeing transfer answers in advance or scoring corrected answers as first answers contaminates the intended comparison. Equivalent defensible study designs may earn the condition point.

21. (1) Record task/version/participant and first prediction plus explanation before output exposure. (2) Record feedback/exposure order and retain the correction separately. (3) Record a novel/delayed response and score, or explicit missingness with reason if known. (4) Replacing a first response with a prompted correction changes what is measured and conceals exposure; a missing response needs the prespecified handling, not an invented score.

Level 7

22. (1) Declare exact triangle inputs and independently derive the reference coordinates by endpoint subtraction. (2) Elicit and retain a prediction before showing results; then change a height or claim. (3) Include a known wrong-sign implementation and require a coordinate check to discriminate it, rather than self-comparison. (4) State that the run checks this example/probe and deliberate fault, with no learner-transfer claim. An algebraic derivation can additionally support the stated telescoping identity.

23. (1) Describe file hashes/census as identity and packaging checks. (2) Describe notebook success as fresh replay of declared cells in a recorded kernel. (3) State that Lean was not rerun locally and identify saved source-bound formal evidence as historical. (4) Explicitly leave full repository integration, full route execution, and learner effects unmeasured/not run as appropriate. Never turn an unavailable tool into a pass.

24. (1) $df = (1/2, -1/3, 3/4, -11/12)$. (2) Residual $(0, 0, 0, 13/12)$, sum $13/12$. (3) Replacing closing claim $1/6$ by $-11/12$ makes the claims equal the displayed differences. (4) A first independently scored unfamiliar-carrier prediction is one proposed transfer observation under declared exposure conditions; one response does not estimate a population effect, validate this rubric, or establish delayed transfer.

Suggested use of scores

Report both the total and eight level subtotals (12 points each). Use missed items to assign arithmetic reconstruction, a new control, a source audit, a theorem-hypothesis reading, or a learner-record revision. Any readiness cutoff is an instructor's choice and must be labeled as such; this package claims no calibrated cutoff. Retain the original score when a learner improves after feedback, and score the new task separately.

Paper 007: Prediction, execution, and learning evidence

Theoretical minimum: Paper 007

AuthorityEffect: None. The Apartment 50 preface is fictional staging. Study the guide, attempt these problems before opening the instructor key, and retain your first answer separately from any correction.

Use exact rational arithmetic. Unless a problem changes it, the carrier has vertices a, b, c, ordered directed edges ab, bc, ca, no filled face, claims $h = (1/3, 1/3, 1/3)$, and potential $f = (0, 1/3, 2/3)$. Define

$$(df)_{st} = f_t - f_s, \qquad r = h - df, \qquad C(x) = x_{ab} + x_{bc} + x_{ca}.$$

A potential adjustment changes the representative r; replacing h changes the claims. A supplied execution-status string is data, not an execution.

There are eight instructional levels, three problems per level, four points per problem: 96 points total. The levels order prerequisites; they are not measured learner abilities or age categories. For every answer give the result, a reason, and its scope. No speed requirement is imposed.

Level 0: Declare the lesson

1. **Carrier and convention.** List the vertices, ordered edges, coefficient system, and difference convention. Does drawing the triangle also declare a filled 2-cell? Identify one operation that changes the carrier rather than only the potential.
2. **Exact input.** Compare `Fraction(1, 3)`, `Fraction("1/3")`, and `Fraction(1/3)`. Which two specify the intended rational input directly? Why is an exact fraction made from a float not automatically the intended rational?
3. **Five components.** Outline a lesson containing a declared finite example, worked calculation, learner prediction, changed-input probe, and evidence boundary. Where must the first prediction be recorded? What does a stored notebook answer fail to record?

Level 1: Compute before running

4. **Worked triangle.** Compute all coordinates of df and r, then $C(h)$, $C(df)$, and $C(r)$. State whether all three claims are potential differences.
5. **Constant shift.** Add $7/5$ to each height. Compute the three shifted heights and predict the new df and r without running a cell. Explain the cancellation.
6. **Changed claims.** Replace h by $-h$, keeping f fixed. Compute the residual and its signed cycle sum. Is this residual equal to $-r$? Explain which quantity was changed.

Level 2: Separate an invariant from a repair

7. **General cycle.** For a directed closed walk $v_0, \dots, v_m = v_0$, prove that $\sum_{i=0}^{m-1}(f_{v_{i+1}} - f_{v_i}) = 0$. Deduce the residual-sum identity for arbitrary rational edge claims on that walk. Does testing three potentials prove the identity for every potential?
8. **Completion.** Keep the first two claims at $1/3$ and choose the third to make the cycle sum zero. Give a potential realizing all three completed claims. Is choosing the missing claim the same operation as adjusting a potential on the original claims?
9. **Path versus cycle.** On the path $a \to b \to c$ with claims $(1, 2)$, use $f = (0, 1, 3)$. Add a closing $c \to a$ claim of 1. Compute the closed-cycle residual and sum. Which closing claim would instead make the table exact? Explain why drawing a filled face does not silently zero the original discrepancy.

Level 3: Try to fool the checker

10. **Wrong-sign mutant.** A faulty implementation uses $\widetilde{df}_{st} = f_s - f_t$. Compute $\widetilde{df}$ and $h - \widetilde{df}$. Does checking only $C(\widetilde{df}) = 0$ and $C(h - \widetilde{df}) = 1$ detect it? Give a predicted-coordinate check that does.
11. **Matching LP objectives.** In the toy linear program minimize the non-negative variable x subject to $x \geq 1$, with dual maximize y subject to $0 \leq y \leq 1$, a checker receives $x = 0, y = 0$. Both computed objectives equal zero. Explain why this is not an optimality certificate, and give the missing feasibility checks and a valid optimal pair.
12. **Status-string grade.** The legacy function counts code cells whose source contains the substring `assert` and treats supplied status `"ok"` as a pass. Evaluate it on:
    - `# assert example` with status `"ok"`;
    - `print("assert")` with status `"ok"`;
    - `assert False` with status `"error"`. Give the reported fraction and explain two reasons it is not a measure of learning or fresh execution.

Level 4: Reproduce the right bytes

13. **Stored outputs versus execution.** A test reads notebook JSON and finds six code cells, execution counts 1 to 6, and the expected output strings. Has it executed the notebook? Describe a fresh replay that compares outputs without overwriting the deposit, and one claim even a successful replay does not support.
14. **Historical source move.** A cited file at commit A is missing locally, while a file with the same name exists at current commit B with different bytes. State a permissible replay strategy, an impermissible substitution, and the information to record if A's bytes cannot be obtained.
15. **Historical 20/20.** The route receipt reports 20 passes dated 2026-09-25; its manifest paths refer to the former notebook tree, and one lesson calls a retired norm API. What can the saved receipt support? What must be maintained and rerun before asserting a fresh full-route pass? Does passing the standalone fraction notebook resolve those route blockers?

Level 5: Read the formal statement

16. **List bound.** The Lean bound assumes $n \in \mathbb{N}$, an integer list h, `h.length n`, and $|x| \leq n - 1$ for every entry; subtraction in `Nat` is truncated at zero. Compute the list mass and ceiling for $n = 3, h = (2, -2, 1)$. Show a permitted list that saturates the ceiling. State the consequence at $n = 0$ and $n = 1$.
17. **Missing hypotheses.** For $n = 3$, give one list violating the per-entry cap and another violating only the length bound, each with mass greater than $n(n-1)$. Explain why neither refutes the conditional theorem and why it cannot be read as a human cognitive limit.
18. **Declaration census.** A saved named report lists 21 declarations; the saved full package-local report lists 64, including generated/private names. Which count answers each scope? Does a whitelist containing `Classical.choice` show that it was used? Does a saved report establish a fresh kernel run on this host? Explain what unary nesting preservation does and does not bound.

Level 6: Assess a person

19. **Threshold fixture.** The legacy test accepts supplied pass rate 92/100, transfer index 88/100, and efficiency 85/100 under chosen thresholds $9/10, 4/5, 4/5$. Calculate which comparisons pass. What participant evidence and definitions are absent? What would be wrong with calling the result a measured 90% learner pass rate?
20. **Transfer comparison.** Propose a comparison of full worked examples with progressively incomplete examples. Specify baseline assessment, assignment/comparison conditions, independently scored novel tasks, and a delayed assessment. Identify one way that answer-key exposure can

contaminate the comparison.

21. **First-response record.** Design a minimal learner record distinguishing first prediction, explanation, output exposure, correction, and later transfer response. How should a missing delayed response be represented? Why should a correction after seeing the answer not replace the first score?

Level 7: Defend the publication's claim

22. **Lesson redesign.** A notebook sets `expected = 1`, assigns `measured = expected`, and asserts equality under the title "all students transfer cycle reasoning." Replace it with a five-component triangle lesson, including an independently derived reference and a control that can detect a specific error. Write its strongest justified evidence sentence.
23. **Receipt audit.** A release verifier reports that file hashes and the two-way file census match. A fraction notebook replay passes. Lean is unavailable. Write a publication-ready verification paragraph distinguishing these outcomes from historical Lean evidence, repository integration, and learner findings.
24. **New four-cycle.** Use ordered edges pq, qr, rs, sp, claims $(1/2, -1/3, 3/4, 1/6)$, and heights $(0, 1/2, 1/6, 11/12)$. Compute df, the residual, and its cycle sum. Choose a replacement closing claim that makes the table exact. State what an independently scored prediction on this unfamiliar carrier would contribute to a proposed transfer assessment, and what one such response cannot establish.

Scoring and remediation

The key gives four explicit points for each problem. Total and level scores locate work to revisit; this rubric is a proposed teaching aid, not a validated certification standard. Treat a wrong coefficient system, silent change of carrier or claims, unrecorded first response, or promotion of a replay into learner evidence as an error in that problem's scope point. Do not erase a correct calculation elsewhere.

Use the guide to repair the corresponding skill, then try a changed instance and retain both attempts. No grade grants authority to publish a learning-effect claim.